Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco · Faculty Mentor: James Andro-Vasko, Ph.D. · Department of Computer Science, University of Nevada, Las Vegas

Summer Undergraduate Research Fellowship (SURF) 2026 · Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLVDockerIDEs

Ready-made course environments:
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

THE HOOK

Week one of a programming course is often lost to installing software, not to programming. Every year, several hundred UNLV students in the introductory C++ course (CS 135) and the x86-64 assembly course (CS 218) spend their first days making a compiler or assembler, a linker, a debugger, and an editor work on dozens of different laptops. Owners of the newest Macs sometimes cannot run the course's 19 GB virtual machine at all and are sent to a campus lab instead. This project asks a simple question: can the whole course environment be one download that behaves the same on the laptops students actually own, with no server, no account, and nothing recorded, and how would we know?

FOUR TERMS USED ON THIS POSTER

Container a sealed, pre-configured copy of a course's tools that runs inside any laptop. Docker is the free program that runs containers.
Image the downloadable file a container starts from. The same 552 MB image was used on every machine in this study.
Translation how an ARM chip (Apple Silicon Macs) runs software built for Intel chips. Correct, but slower.
Debugger a tool that pauses a running program so students can look inside it. Some CS 218 assignments require it.

Abstract

Introductory programming courses often lose their first week to software installation, not programming. In UNLV's introductory C++ course (CS 135) and x86-64 assembly course (CS 218), several hundred students a year must get a compiler or assembler, a linker, a debugger, and an editor working on dozens of different laptops, and students with Apple Silicon Macs often cannot run the course's 19 GB virtual machine at all. This project packages each course's exact toolchain and a browser-based Visual Studio Code editor into a Docker container that runs entirely on the student's own laptop: one 552 MB download, one command, no server, no account, and nothing recorded. Coursework stays in an ordinary folder, so the container can be deleted or updated without touching student files. To test whether that promise holds on the laptops students actually own, one unmodified script performed a student's first day and four real CS 218 assignments, including a multithreaded program, on four machines: Intel and ARM Linux rented from AWS, Windows 11 rented from Azure, and an M1 Pro MacBook Pro. All four downloaded the identical image and passed every assignment. The editor opened in 0.5 s on Intel chips and 5 to 7 s on ARM chips, where x86-64 code runs through a translator roughly ten times slower but still within seconds. The one limit is the debugger, which the translator cannot support on ARM. That boundary is documented in the student handout, every raw result is public, and a classroom study is planned next.

Introduction and background

The problem is old and well documented. Other universities measured 75–100 minutes of class time per semester lost to installs [6], found that most of their help traffic was environment mismatch rather than course content [3], and reported students with M1/M2 MacBooks unable to run the course's virtual machine at all [4]. Institutions have answered in four ways, and UNLV has lived through the first two:

1 Shared login servers (UNLV, 2000s–2024)
Every student logged in to a department computer over the network, usually with PuTTY, even from lab machines. The servers are retired and their documentation was deleted in a 2025 website rebuild [12].

2 A virtual machine per course (CS 218 today)
A 19 GB copy of a whole computer that runs inside the laptop. Slow to boot, and so slow on Apple Silicon Macs that the course sends M1/M2 owners to the lab with a USB drive.

3 Cloud editors (Harvard CS50 [1])
The environment lives on a company's servers, used by more than 700,000 people. Needs internet and an account, and students finish the course never having set up their own machine.

4 Monitored platforms (CodeDive, South Korea [8])
The university's cluster runs the editor and records every command plus a code snapshot after every one-second pause in typing: 24,845 snapshots from 95 students in two weeks.

Problem and research focus

Before the fellowship I built a fifth option: a container holding each course's exact tools plus the VS Code editor, running entirely on the student's own laptop and opened in a browser tab. One download, one command, no server, no account, nothing recorded, and coursework kept in a normal folder that survives deleting the container. It worked on the laptop I built it on. A promise tested only where it succeeds is an advertisement, not a result, so the research question is whether it survives contact with the laptops students actually own:

Q1 Does the same package behave identically on Windows, macOS, and Linux, on Intel and ARM chips, including on real CS 218 assignments?
Q2 What does it cost the student: download size, start-up time, memory, build time?
Q3 Where does it break, and can we say exactly why?

Methodology

What we built

Two containers, one per course, each built from Ubuntu Linux and holding the browser-served VS Code editor plus the exact tool versions the instructor chose: the g++ compiler for CS 135; the yasm and nasm assemblers, the linker, and the gdb debugger for CS 218. The editor listens only on the student's own machine, on a port numbered after the course, with every AI feature disabled per course policy. A student installs Docker once, runs the one command below, and opens a browser tab. Coursework stays in a normal folder on the laptop, never inside the container, so the container can be deleted or updated at any time (Figure 1).

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace seancnc/unlv-x86-ide
```

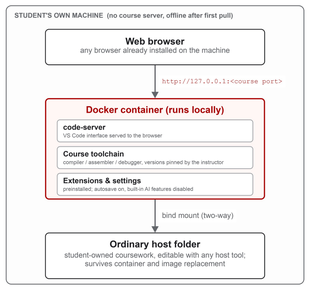


Figure 1. Everything runs on the student's machine. The only time the internet is needed is the first download of the image.

How we tested it: same exam, four kinds of computer

We cannot borrow one of every laptop students own, so we rented them. Three blank machines came from Amazon's and Microsoft's clouds for under $10 in total, so each could play the role of a fresh student laptop. The fourth was a 2021 14-inch MacBook Pro with an M1 Pro chip and 16 GB of memory, the one machine we owned rather than rented.

Windows 11 PC	Linux PC	Apple Silicon Mac	Linux on ARM
Intel chip, 16 GB rented (Azure)	Intel chip, 8 GB rented (AWS)	M1 Pro, 16 GB owned (2021 MacBook Pro)	ARM chip, 8 GB rented (AWS)

One script, run unchanged on all four

1 Download the image and record its fingerprint, confirming it is the identical file on every machine.
2 Start it and time how long until the editor answers in the browser; check that starter files appear exactly once.
3 Write, assemble, link, and run a program three times; run a scripted debugger session and record whether it works.
4 Build and run four real CS 218 assignments spanning the course, including a multithreaded one with a checkable answer, and sample peak memory.
5 Delete the container, recreate it, and check that the student's files survive.

The script writes every number to a file. Nothing on this poster was typed by hand; the raw files are archived in the public repository.

Results

4 of 4
tested configurations ran all four selected CS 218 assignments correctly, including the multithreaded one, on the identical image

0.5 s
until the editor opens on Intel chips; about 5 s on Apple Silicon. A course virtual machine takes minutes.

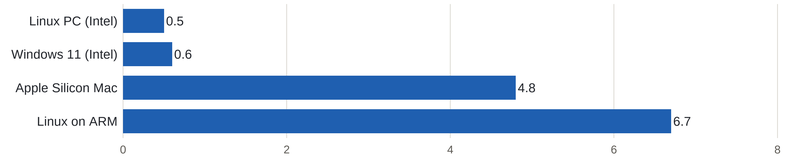
552 MB
one-time download, the identical file on every machine, versus a 19 GB virtual machine

Table 1. What the script recorded on each machine (memory in MiB).

Measure	Windows 11 (Intel)	Linux PC (Intel)	Apple Silicon Mac	Linux on ARM
All four assignments correct	yes	yes	yes	yes
Files survive deleting the container	yes	yes	yes	yes
Debugger	works	works	**silent**	**silent**
Editor ready in (seconds)	0.6	0.5	4.8	6.7
Memory while idle	56	54	263	255
Peak memory during coursework	90	n/a	358	334
Heaviest assignment builds in (s)	0.6	0.3	3.1	3.6
Checks passed / failed	13 / 0	13 / 0	12 / 0	12 / 0

ARM totals exclude the debugger check: under translation the script records the debugger result as a finding rather than a pass or fail, so 12 checks are counted instead of 13.

Figure 2. Seconds until the editor is ready



What the numbers mean. On Intel and AMD machines the cost is close to zero: the editor is ready in about half a second, idles at roughly 55 MiB, and the heaviest assignment builds in under a second. Windows 11 was the strongest host in the matrix. On ARM chips every assignment still produced the correct answer; startup rose to five to seven seconds, peak memory to about 350 MiB (a small slice of an 8 GB laptop), and compile-heavy steps ran roughly ten times slower, yet every build finished in under four seconds. Because the identical 552 MB image was downloaded everywhere, any difference between machines is due to the machine, not the package.

The one limit, in plain words. The assembly course teaches the Intel-style x86-64 instruction set, so that package is built for Intel chips on purpose, and ARM chips run it through a translator. The translator cannot support the debugger: a debugger run on a Mac appears to succeed but prints no register values, so the student handout tells Apple Silicon owners not to trust it and to finish debugger assignments on an Intel machine or in the lab. We also found that the translator that ships with Ubuntu Linux crashes while Docker's own does not; that fix is one line in the handout. The boundary is a translator boundary, not an operating-system boundary: Windows and Intel machines get everything.

Concluding discussion: why this matters

For students

Week one goes to learning, not installing. The same environment on every tested configuration, offline after one download, with coursework in a normal folder that the student owns and keeps after the course ends. Nothing is recorded or sent anywhere. All four tested configurations ran the selected assignments correctly; debugging remained unavailable on ARM, and the handout says so.

For the course and the department

No servers to maintain, no VPN, no 19 GB image to rebuild each semester. The instructor updates one recipe and every student gets the same tools, built and published automatically. A trade-off nobody had measured is now explicit: keep the real x86-64 instruction set and lose the debugger on ARM, until a hybrid container brings it back.

For other courses and researchers

The rent-four-computers, one-script method is course-agnostic and could vet any course environment, and every raw result is public so anyone can check the numbers. The design sits between cloud editors and monitored platforms: the same convenience without the account, the internet dependence, or the surveillance, enforced by architecture rather than by policy.

What comes next

A classroom study in CS 135 and CS 218 with IRB approval: how many students reach a working setup without help, how much staff time it saves, and how students feel about it, using instruments adapted from [3, 4, 6]. A three-student high-school pilot has already run with no setup questions at all. Then a hybrid assembly container for ARM that runs the editor natively and translates only student programs, which could bring the debugger back to Apple Silicon.

References (numbered as in the written report). [1] D. J. Malan. Containerizing CS50: Standardizing Students' Programming Environments. ITiCSE 2024. [3] S. Valstar, W. G. Griswold, L. Porter. Using DevContainers to Standardize Student Development Environments. ITiCSE 2020. [4] K. Fernalld, T. OConnor, S. Sudhakaran, N. Nur. Lightweight Symphony: Reducing CS Student Anxiety with Standardized Docker Environments. SIGITE 2023. [6] D. P. Harvie, J. R. Cody, C. Morrell, T. T. Estes. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019. [8] H. Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 15(19), 2025. [12] UNLV Dept. of Computer Science. Student Center: Remote Access and File Storage. tux.cs.unlv.edu, accessed Aug. 2026.

Supported by the UNLV Office of Undergraduate Research Summer Undergraduate Research Fellowship. Every number on this poster comes from the published test script's output files, archived in the repository under tests/record-results.